

Midterm Exam Solution (10.24, Friday)

Fall 2025

Answer #1

```
[[1, 9], [3, 10]]  
[[1, 9], [3, 4]]
```

Answer #2

```
1  
3
```

Answer #3

```
(2, 3, 'hello') 1
```

Answer #4

```
add_lambda = lambda x, y : x+y
```

Answer #5

```
class Adder:  
    def __call__(self, x, y):  
        return x + y
```

Answer #6

```
Bulldog  
Kaw
```

Answer #7

```
6  
4  
2
```

Answer #8

```
[0, 2, 4]
```

Answer #9

```
Access denied!  
User deleted by admin
```

Answer #10

[0, 1, 4, 9, 16]

0

1

Answer #11

[[0 1 2]

[3 4 5]]

Answer #12

20

Answer #13

[1 2]

[1 3]

Answer #14

(3, 2)

(2, 2, 3)

Answer #15

torch.Size([5, 2])

Answer #16

For a 2D convolution, the spatial output size (height and width) is computed by

Output size = $(32 + 2*1 - 4)/2 + 1 = 16$. So the output height and width are 16 x 16.

Answer #17

The batch size is 20. The output channel is 5. The spatial output size is given by

Output size = $(30 - 6 + 2*2) / 2 + 1 = 14+1 = 15$.

Thus, y.shape is torch.Size([20, 10, 15, 15]).

Answer #18

(a) `torch.Size([10, 2, 4, 2])`

(b) Advantages of using Max Pooling in CNN are given below (at least two advantages are presented in the answer sheet):

(1) Dimensionality Reduction:

Max Pooling reduces the spatial dimensions (height and width) of the feature maps. This decreases the number of parameters and computations in the network, which helps make the model more efficient (= faster) and less prone to overfitting.

(2) Translation Invariance:

Max pooling allows the network to detect features regardless of small translations or distortions in the input image. This means that even if a feature (like an edge or texture) shifts slightly, it can still be recognized. Thus, Max pooling help CNNs become more robust and less sensitive to small translational variations in the input data.

Answer #19

```
tensor([[[[ 5,  8],
           [-1,  7]]]])
```

Answer #20

Data: [0, 1], Label: [0, 2]

Data: [2, 3], Label: [4, 6]

Data: [4], Label: [8]

Answer #21

```
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(1, 16)
        self.dropout = nn.Dropout(p=0.5)
        self.bn1 = nn.BatchNorm1d(16)
        self.fc2 = nn.Linear(16, 1)
        self.bn2 = nn.BatchNorm1d(1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = torch.relu(self.bn1(self.fc1(x)))
        return self.fc2(x)
        x = self.bn2(self.fc2(x))
        return x
```

Answer #22

(1) Dropout

During training, Dropout randomly drops (sets to zero) a certain percentage of neurons in a layer at each forward pass.

- The key idea of dropout is to ensure the distribution of neuron outputs remains consistent between training and testing.

(2) Batch Normalization (BN)

For each mini-batch, BN normalizes the activations of a layer to have zero mean and unit variance, then rescales and shifts them with learnable parameters (γ and β).

- Since BN normalizes the distribution of the pre-activation values (per feature) across the batch, the performance of BN depends on the mini-batch size (disadvantage).

(3) Layer Normalization (LN)

Instead of normalizing across the batch dimension, LN normalizes across the features for each individual sample.

- Since LN normalizes the distribution of the pre-activation values (per sample) across features, the performance of LN does not depends on the mini-batch size (advantage).

Answer #23

Let us consider 1D CNN and its Kernel size is 3. When dilation = 20, the size of receptive field is given by

$$2 \cdot 20 + 1 = 41 \text{ (see Figure 1 carefully)}$$

Answer #24

(1) (3 points) Channel mismatch in conv2 (will crash at runtime).

Fix: set in_channels=16 as follows:

```
self.conv2 = nn.Conv2d(in_channels=16, out_channels=32,
                        kernel_size=3, padding=1)
```

(2) (14 points) Remove the ReLU to the final classification output.

Wrapping it with F.relu squashes negatives and is incompatible with nn.CrossEntropyLoss (which expects raw logits).

Fix: remove the ReLU on fc2 as follows:

```
x = self.fc2(x) # no activation here
```

(3) (3 points) Input size mismatch in fc2 (will crash at runtime).

Fix: use nn.Linear(128, 10) as follows:

```
self.fc2 = nn.Linear(128, 10)
```